



Dart Fundamentals

CS442 | Fall 2026

1

COURSE ROADMAP

Three Sessions to Dart Fluency

1

Dart Syntax,
Types & Control Flow

- Hello Dart & program anatomy
- Types, variables & null safety
- Operators, if/switch, loops
- Practice & common pitfalls

2

Functions,
Collections & Errors

- Parameters & arrow functions
- Closures & higher-order functions
- List / Map / Set in depth
- Generics & exception handling

3

Object-Oriented
Dart

- Classes, constructors & inheritance
- Abstract classes, interfaces & mixins
- Enums, extensions & operators
- OOP best practices

2



SESSION 1

Dart Syntax, Types & Control Flow

- Why Dart, and how to run it (DartPad & CLI)
- Your first program & the anatomy of a function
- Comments, strings, and the core SDK libraries
- Types, variables, and sound null safety

3

SESSION 1 AGENDA

What We'll Cover Today

- 1 Why Dart, and how to run it (DartPad & CLI)
- 2 Your first program & the anatomy of a function
- 3 Comments, strings, and the core SDK libraries
- 4 Types, variables, and sound null safety
- 5 Operators — arithmetic, null-aware & cascade
- 6 Control flow: if/else & switch
- 7 Loops and collection literals
- 8 Hands-on practice & common pitfalls

Session 1 — Dart Syntax, Types & Control Flow

1

4

MOTIVATION

Why Dart?

- Purpose-built by Google as Flutter's UI language
- Compiles to native ARM/x64 code (AOT) for fast apps
- Also runs just-in-time (JIT) — powers Hot Reload
- Sound null safety: null-related bugs caught at compile time
- Familiar C-style syntax — fast to learn if you know Java/JS/C#

GETTING SET UP

Running Dart: DartPad & the CLI

- DartPad (dartpad.dev) — browser sandbox, zero install
- Full SDK install: includes dart, pub, and dartfmt
- `$ dart run file.dart` — compiles & runs a script
- `$ dart format .` — auto-formats code to style guide
- `$ dart analyze` — static analysis & linting

2.1 HELLO, DART!

Your First Dart Program

```
void main() {
  print('Hello, Dart!');
}
```

- Save as hello_world.dart, then run: \$ dart run hello_world.dart
- Output: Hello, Dart!
- Every statement ends in a semicolon — no exceptions

FIGURE 2.1

Anatomy of a Dart Function

```
void main() { }
```

void

Return type — no value returned

main

Function name

()

Marks a function declaration

{ }

Wraps the function's code block

PARAMETERS

main() and Command-Line Arguments

```
void main(List<String> args) {
  if (args.isEmpty) {
    print('No name given!');
  } else {
    print('Hello, ${args[0]}!');
  }
}
```

- \$ dart run hello.dart Nora → Hello, Nora!
- args is a List<String> — one entry per CLI argument

2.2.2 COMMENTS

Documenting Your Code

```
// Inline comment — explains one line
/* Block comment — rarely used in Dart */

/// Doc comment — documents the API below
/// Returns the sum of [a] and [b].
int add(int a, int b) => a + b;
```

- Triple-slash /// comments feed the dartdoc tool
- Square brackets [param] link to symbols in generated docs
- Every public Flutter widget is documented this way

2.1.2 GREETINGS

Refactoring to Functions & Lists

```
void helloDart(String name) {
  print('Hello, $name');
}
List<String> greetings = ['World', 'Mars', 'Oregon'];
for (var name in greetings) {
  helloDart(name);
}
```

- Typed parameters: String name goes inside the ()
- List<String> holds an ordered collection of Strings
- for-in walks every item; \$name interpolates a value

STRINGS

Strings in Depth

```
var a = 'Hello, $name!'; // interpolation
var b = 'Total: ${price * qty}';
var c = r'C:\new\folder'; // raw string
var d = '''Multi-linetext.'''
```

- Common methods: toUpperCase(), trim(), split(), contains()
- isEmpty / isNotEmpty avoid null-vs-empty confusion
- Strings are immutable — every method returns a new String

2.1.3 I/O

I/O & the Dart SDK Libraries

```
import 'dart:io';  
void main() {  
  stdout.writeln('Greet somebody');  
  String input = stdin.readLineSync();  
  helloDart(input);  
}
```

- dart:core loads automatically; others need import
- dart:math, dart:convert, dart:async — common SDK libraries
- dart:html is unavailable in Flutter — different environment, different libraries

2.2.1 TYPES

Dart's Type System

```
int greeting = 'hello'; // compile-time error
```

- Dart is a statically, strictly typed language
- Every variable and function should declare a type, or void
- Type errors are caught at compile time — before your app runs
- Contrast: JavaScript/Python/Ruby infer types only at runtime
- Fewer runtime crashes, because mismatched types never ship

BUILT-IN TYPES

Core Types at a Glance

Type	Example	Notes
int	42, -7	Whole numbers, arbitrary precision
double	3.14, 2.0	64-bit floating point
num	int or double	Common supertype of both
bool	true, false	No truthiness – must be literal bool
String	'hi', "hi"	UTF-16, immutable
List<T>	[1, 2, 3]	Ordered, indexable collection
Map<K,V>	{'a': 1}	Key→value pairs
Set<T>	{1, 2, 3}	Unordered, unique elements

2.2.3 VARIABLES

var, final, const & static

```
var name = 'Nora'; // inferred: String
const pi = 3.14159; // OK: compile-time known
final now = DateTime.now(); // OK: runtime, once
```

- var — type is inferred once, then locked in
- final — assigned exactly once, often in a constructor
- const — compile-time constant, always the same value
- static — belongs to the class itself, not an instance

NULL SAFETY

Sound Null Safety Essentials

```
String name = 'Nora'; // never null
String? nickname;      // nullable, defaults null
print(nickname!.length); // ! asserts non-null
late String description; // set before first use
```

- A plain type (String) can never hold null — enforced at compile time
- Add ? to allow null: String?, int?, List<String>?
- ! (bang) tells the compiler 'trust me, this isn't null' — risky!

TABLE 2.1

Dart Operators

Category	Operators
Arithmetic	* / % ~/ + -
Relational / type test	>= > <= < as is is!
Equality	== !=
Logical	&&
Assignment	= += -= ??=
Cascade	.. ?..

~/ is integer division (rounds down): 5 ~/ 2 == 2 — cascade (..) chains calls on one object

OPERATORS, SIMPLIFIED

Two Operators You Haven’t Used Yet

~/ — integer (whole-number) division

```
int apples = 17;
int perBasket = 5;

print(apples ~/ perBasket);
// 3 — full baskets only
print(apples % perBasket);
// 2 — apples left over
```

is / is! — checking a value’s type

```
Object input = 'twenty-one';

if (input is String) {
  print('This is text!');
}

if (input is! int) {
  print('Not a number.');
```

- ~/ divides then drops the remainder — like counting full baskets
- is checks a value’s real type while the program runs; is! is its opposite
- Both is and is! return a bool, ready to use in an if

OPERATORS, SIMPLIFIED

Give It a Default, Then Chain Some Calls

??= — assign only if null

```
int? highScore;

highScore ??= 0;
// was null, so now it's 0
highScore ??= 100;
// already set — no change

print(highScore); // 0
```

.. and ?.. — cascade notation

```
var cart = <String>[]
  ..add('Pencil')
  ..add('Eraser');
print(cart); // [Pencil, Eraser]

List<String>? maybeCart;
maybeCart?.add('Pen');
// skipped — maybeCart is still null
```

- ??= only assigns when the variable is currently null
- .. lets you call several methods on one object without repeating its name
- ?.. is the null-safe cascade — it skips the whole chain instead of crashing